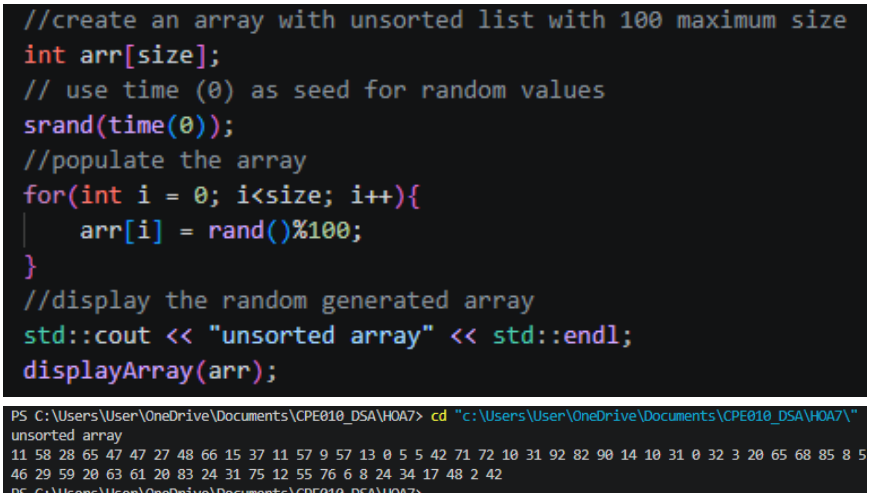
COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | **CPE 010 - Data Structures and Algorithms** | **SECTION:** | **CPE21S2** |
| **DATE PERFORMED:** | **09-02-26** | **DATE SUBMITTED:** | **09-08-26** |
| **NAME:** | **Christian Lee C. Bandigan** | **INSTRUCTOR:** | **Engr. Jimlord M. Quejado** |

**ACTIVITY NO. 7** **Hands-on Activity 7.1 Sorting Algorithms Pt1**

## 1. PROCEDURE OUTPUT

| | |
|---|---|
| Code + Console Screenshot |  |
| Observations | We created an array of one hundred integers by using srand with time(0) as the seed. We used a for loop to insert the random integers under 100 into the array. |

```
//create an array with unsorted list with 100 maximum size
int arr[size];
// use time (0) as seed for random values
srand(time(0));
//populate the array
for(int i = 0; i<size; i++){
    arr[i] = rand()%100;
}
//display the random generated array
std::cout << "unsorted array" << std::endl;
displayArray(arr);
```

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA7> cd "c:\Users\User\OneDrive\Documents\CPE010_DSA\HOA7\"
unsorted array
11 58 28 65 47 47 27 48 66 15 37 11 57 9 57 13 0 5 5 42 71 72 10 31 92 82 90 14 10 31 0 32 3 20 65 68 85 8 5
46 29 59 20 63 61 20 83 24 31 75 12 55 76 6 8 24 34 17 48 2 42
```

Table 7-1. Array of Values for Sort Algorithm Testing

| | |
|---|---|
| Code + Console Screenshot |  |

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    //step 1L for i = 0 to N-1 repeat inner loop
    for(int i = 0; i <arrSize; i++){
        //step 2 inner loop that foe K = i+1 to N-i repeat
        for(int j = i+1; j < arrSize; j++){
            //step 3: swap if the arr[j] > arr[i]
            if(arr[i] > arr[j]){
                std::swap(arr[i],arr[j]);
            }
        }
    }
}
```

| | |
|---|---|
| | PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA7> cd "c:\Users\User\OneDrive\Documents\CP<br>unsorted array<br>59 37 17 37 22 67 97 83 63 14 96 59 76 46 25 87 52 98 68 45 14 44 57 53 6 82 7 82 11 32 96 9<br>10 83 64 79 91 42 97 2 52 39 93 31 49 56 28 96 93 35 5 39 61 37 75 15 58<br>bubble sort<br>2 5 6 6 7 7 9 10 10 10 11 13 14 14 15 15 16 17 17 19 20 20 22 25 27 28 31 32 35 37 37 37 38<br>82 82 83 83 84 85 87 89 90 90 91 92 92 93 93 93 96 96 96 97 97 98 99 99 |
| Observations | Bubble sort works by comparing adjacent elements and swaps them if they are in the wrong order. It repeatedly moves larger values towards the end after each pass.<br>Since this algorithm use a nested for loop its time complexity are:<br>Best-Case Time Complexity: $O(n^2)$<br>Worst-Case Time Complexity: $O(n^2)$ |

Table 7-2. Bubble Sort Technique

| | |
|---|---|
| Code + Console Screenshot | 20 template <typename T><br>21 //array, current position, size of array<br>22 int routineSmallest(T arr[],int K, const int arrSize){<br>23 int position;<br>24 T smallestElem = arr[K];<br>25<br>26 for(int j = K+1; j <arrSize; j++){<br>27 if(arr[j] < smallestElem){<br>28 smallestElem = arr[j];<br>29 position = j;<br>30 }<br>31 }<br>32 return position;<br>33 }<br><br>35 template <typename T><br>36 void selectionSort(T arr[], size_t arrSize){<br>37 int POS, temp, pass;<br>38 for(int i = 0; i < arrSize; i++){<br>39 POS = routineSmallest(arr, i, arrSize);<br>40 //swapping:<br>41 temp = arr[i];<br>42 arr[i] = arr[POS];<br>43 arr[POS] = temp;<br>44 //verification and counting the number of passes that was done;<br>45 pass++;<br>46 }<br>47 }<br><br>PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA7> cd "c:\Users\User\OneDrive\Documents\CPE010_DSA\HOA7<br>unsorted array<br>43 76 36 93 86 44 36 54 79 20 28 91 14 84 31 73 21 60 1 7 23 29 95 31 80 24 75 75 31 61 34 81 69 30 84 65<br>0 49 17 55 10 40 17 16 89 30 65 56 79 48 38 88 11 31 5 94 98 54 60 88 93<br>selection sort<br>1 4 5 7 7 9 9 10 11 14 14 15 15 16 17 17 20 20 21 21 93 23 23 23 24 24 25 28 29 30 30 31 31 31 31 34 34 3<br>76 79 79 79 80 81 82 84 84 86 86 88 88 88 91 89 91 91 92 93 93 94 98 95 |
| Observations | Selection sort works by finding the smallest element in each iteration and |

| | |
|---|---|
| | placing it at the left side of the array. The code uses two for loops, one for finding the smallest element in the array and one that swaps the elements. That's why the time complexity for this algorithm is:<br>Best-Case Time Complexity: $O(n^2)$<br>Worst-Case Time Complexity: $O(n^2)$ |

Table 7-3. Selection Sort Algorithm

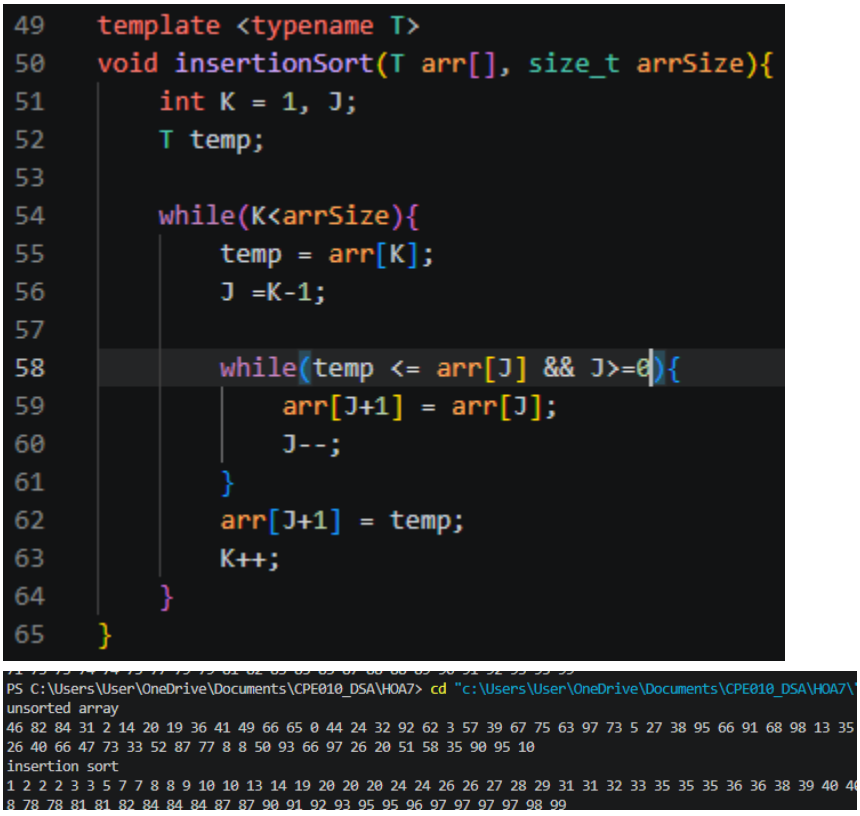
| | |
|---|---|
| Code + Console Screenshot |  |
| Observations | Insertion sort works like sorting playing cards where you build a sorted portion on the left. It takes one element at a time and inserts it into its correct position. This algorithm uses a nested loop so its time complexity is $O(n^2)$ but since it uses a while loop if the array is already sorted the inner loop doesn't run so its best time complexity is O(n).<br>Best-Case Time Complexity: O(n)<br>Worst-Case Time Complexity: $O(n^2)$ |

Table 7-4. Insertion Sort Algorithm

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**Pseudocode**

Begin

INITIALIZE size = 100
INITIALIZE candidates = 5
CREATE an array with 100 elements
CREATE an array votes with 5 elements
FOR i = 0 to 99
    GENERATE a random number from 1 to 5
    INPUT the number in arr[i]
END FOR
DISPLAY Unsorted Array
SORT  array using Insertion Sort
DISPLAY Sorted Array
COUNT Votes
DISPLAY the vote count of each candidate
FIND most votes
INPUT the returned index in winner
IF winner is 0
   Display Bo Dalton Capistrano
ELSE IF winner is 1
    Display Cornelius Raymon Agustin
ELSE IF winner is 2
    Display Deja Jayla Banaga
ELSE IF winner is 3
    Display Lalla Brielle Yabut
ELSE IF winner is 4
    Display Franklin Relano Castro
END IF

END

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int size = 100;
const int candidates = 5;
void displayArray(int arr[]);
void countVotes(int arr[], int votes[]);
int mostVotes(int votes[]);

int main(){
```

```
    int votes[candidates];
    int arr[size];
    srand(time(0));
    for(int i = 0; i<size; i++){
        arr[i] = (rand()%5) + 1;
    }
    std::cout << "Unsorted Array:" << std::endl;
    displayArray(arr);

    insertionSort(arr,size);
    std::cout << "Sorted Array:" << std::endl;
    displayArray(arr);

    countVotes(arr,votes);
    std::cout << "Vote Count" << std::endl;
    std::cout <<"Bo Dalton Capistrano: "<< votes[0] << " votes\n";
    std::cout <<"Cornelius Raymon Agustin: "<< votes[1] << " votes\n";
    std::cout <<"Deja Jayla Banaga: "<< votes[2] << " votes\n";
    std::cout <<"Lalla Brielle Yabut: "<< votes[3] << " votes\n";
    std::cout <<"Franklin Relano Castro: "<< votes[4] << " votes\n";

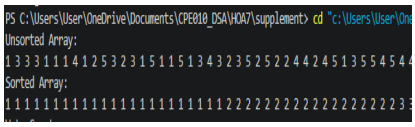
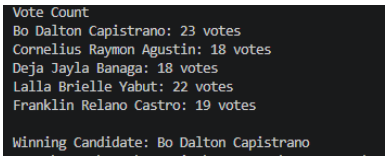
    int winner = mostVotes(votes);
    switch(winner){
        case 0:
            std::cout << "\nWinning Candidate: Bo Dalton Capistrano"
<<std::endl;
            break;
        case 1:
            std::cout << "\nWinning Candidate: Cornelius Raymon Agustin"
<<std::endl;
            break;
        case 2:
            std::cout << "\nWinning Candidate: Deja Jayla Banaga"
<<std::endl;
            break;
        case 3:
            std::cout << "\nWinning Candidate: Lalla Brielle Yabut"
<<std::endl;
```

```
            break;
        case 4:
            std::cout << "\nWinning Candidate: Franklin Relano Castro"
<<std::endl;
            break;
    }
    return 0;
}

void displayArray(int arr[]){
    for(int i = 0; i<size; i++){
        std::cout << arr[i] <<" ";
    }
    std::cout << std::endl;
}

void countVotes(int arr[], int votes[]){
    int instance;
    for(int i = 0; i < candidates;i++){
        instance = 0;
        for(int j = 0; j<size; j++){
            if(arr[j] == i + 1){
                instance++;
            }
        }
        votes[i] = instance;
    }
}

int mostVotes(int votes[]){
    int largest = 0;
    for(int j=1;j<candidates;j++){
        if(votes[j]> votes[largest]){
            largest = j;
        }
    }
    return largest;
}
```

| Output Console Showing Sorted Array | Manual Count | Count Result of Algorithm |
|---|---|---|
|  | Candidate 1 = 23 votes<br>Candidate 2 = 18 votes<br>Candidate 3 = 18 votes<br>Candidate 4 = 22 votes<br>Candidate 3 = 19 votes |  |

For this problem I used the insertion sort algorithm because it's easier to understand, has the time complexity of $0(n^2)$, has the best-case time complexity of O(n) when the array is already sorted, and works perfectly for an array of 100 elements. I would say my developed counting algorithm is effective because it correctly counts the number of votes but it still needs improvement because I didn't really maximize the sorted array.

**Conclusion**

In this activity I learned about the different sorting algorithms specifically bubble sort, insertion sort, and selection sort. I was able to implement each of these sorting algorithms in C++ to sort an array of one hundred elements. Moreover, in the supplementary activity, I was able to solve a problem by identifying and implementing the best sorting algorithm for this specific problem.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.